1.6 Logistic regression

- some regression algorithms can be used for classification (and vice versa)
- *logistic regression* (also called *logit regression*) is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that this email is spam?)
- if the estimated probability is greater than 50%, then the model predicts that the instance belongs to that class (called the *positive class*, labeled “1”), and otherwise it predicts that it does not (i.e., it belongs to the *negative class*, labeled “0”)
- this makes it a *binary classifier*

1.6 Logistic regression

- just like a linear regression model, a logistic regression model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly, like the linear regression model does, it outputs the *logistic* of this result:

$$\hat{p} = \sigma(\boldsymbol{w}^T \boldsymbol{x}).$$

- the logistic, denoted $\sigma(\cdot)$, is the *sigmoid function* (i.e., S-shaped) that outputs a number between 0 and 1; it is defined as:

$$\sigma(t) = \frac{1}{1 + \exp(-t)},$$

and its graph is given in Figure 18

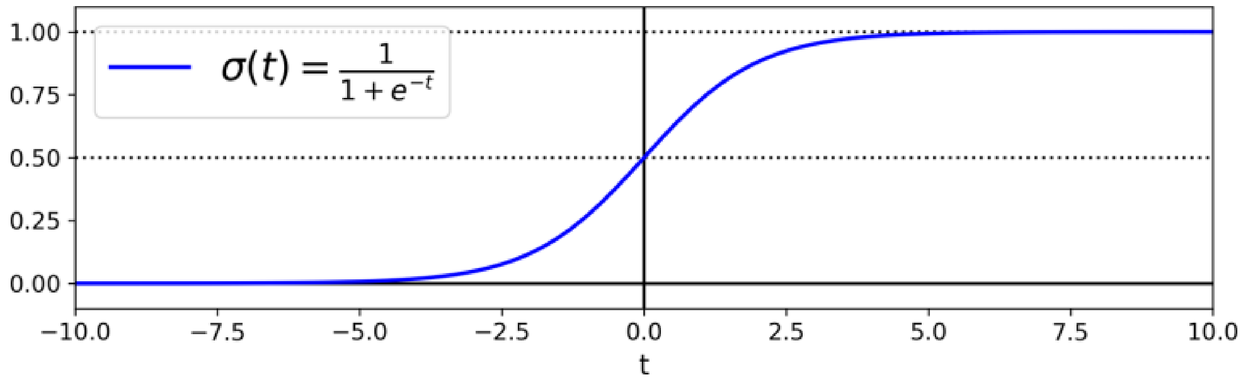


Figure 18: Sigmoid function.

1.6 Logistic regression

- once the logistic regression model has estimated the probability $\hat{p}$ that an instance $\boldsymbol{x}$ belongs to the positive class, it can make its prediction $\hat{y}$ easily as:

$$\hat{y} = \begin{cases} 0, & \text{if } \hat{p} < 0.5 \\ 1, & \text{if } \hat{p} \geq 0.5 \end{cases}.$$

- notice that $\sigma(t) < 0.5$ when $t < 0$, and $\sigma(t) \geq 0.5$ when $t \geq 0$, so a logistic regression model predicts 1 if $\boldsymbol{w}^T \boldsymbol{x}$ is positive and 0 if it is negative
- the score t is often called the *logit*; the name comes from the fact that the *logit function*, defined as $\text{logit}(\hat{p}) = \log(\hat{p}/(1-\hat{p}))$, is the inverse of the sigmoid function
- indeed, if we compute the logit of the estimated probability $\hat{p}$, we will find that the result is t
- the logit is also called the *log-odds*, since it is the log of the ratio between the estimated probability for the positive class and the estimated probability for the negative class

1.6 Logistic regression

- for training the logisitic regression, the objective is to set the weight vector $\boldsymbol{w}$ so that the model estimates high probabilities for positive instances ($y = 1$) and low probabilities for negative instances ($y = 0$)
- this idea is captured by the loss function shown below, for a single training instance $\boldsymbol{x}$:

$$l(\boldsymbol{w}) = \begin{cases} -\log \hat{p}, & \text{if } y = 1 \\ -\log(1 - \hat{p}), & \text{if } y = 0 \end{cases}.$$

- this loss function makes sense because $-\log t$ grows very large when t approaches 0, so the loss will be large if the model estimates a probability close to 0 for a positive instance, and it will also be very large if the model estimates a probability close to 1 for a negative instance
- on the other hand, $-\log t$ is close to 0 when t is close to 1, so the loss will be close to 0 if the estimated probability is close to 0 for a negative instance or close to 1 for a positive instance, which is precisely what we want

1.6 Logistic regression

- it can be also written as:

$$l(\boldsymbol{w}) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

- the loss function over the whole training set is the average loss over all training instances
- it can be written in a single expression called the *log loss*, as:

$$\mathcal{L}(\boldsymbol{w}) = -\frac{1}{n} \sum_{i=1}^{n} \left[y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]. \tag{12}$$

- the bad news is that there is no known closed-form equation to compute the value of $\boldsymbol{w}$ that minimizes this loss function (there is no equivalent of the normal equation)
- the good news is that this loss function is convex, so gradient descent (or any other optimization algorithm) is guaranteed to find the global minimum (if the learning rate is not too large and we wait long enough)

1.6 Logistic regression

- the gradient of the loss function is given by:

$$\nabla_{\boldsymbol{w}} \mathcal{L}(\boldsymbol{w}) = \frac{1}{n} \boldsymbol{X}^{\top} (\sigma(\boldsymbol{X}\boldsymbol{w}) - \boldsymbol{y}).$$

- this equation looks very much like equation (8): for each instance it computes the prediction error and multiplies it by the feature values, and then it computes the average over all training instances
- once we have the gradient vector containing all the partial derivatives, we can use it in the batch gradient descent algorithm
- for stochastic GD, we would take one instance at a time, and for mini-batch GD we would use a mini-batch at a time
- just like the other linear models, logistic regression models can be regularized using ℓ_1 or ℓ_2 penalties